บันทึกสรุปเชิงวิชาการ

ตารางแฮชแบบขนานขนาดใหญ่บนหน่วยประมวลผลกราฟิก

แนวคิด การออกแบบ และสมรรถนะของ cuCollections

แหล่งที่มาที่สรุป

Daniel Juenger, Nico Iskos, Yunsong Wang, Jake Hemstad, Christian Hundt และ Nikolay Sakharnykh. (2023, 6 มีนาคม). *Maximizing Performance with Massively Parallel Hash Maps on GPUs*. NVIDIA Technical Blog.

`https://developer.nvidia.com/blog/maximizing-performance-with-massively-parallel-hash-maps-on-gpus/`

สืบค้นเมื่อ 3 พฤศจิกายน 2026 (2026-11-03)

เอกสารฉบับนี้เป็น**บันทึกสรุปเพื่อการศึกษา**ที่เรียบเรียงขึ้นใหม่ด้วยถ้อยคำและโครงสร้างของผู้จัดทำ **มิใช่คำแปล**ของบทความต้นฉบับ ลิขสิทธิ์ของบทความต้นฉบับเป็นของ NVIDIA Corporation ผู้อ่านควรศึกษาต้นฉบับเพื่อรายละเอียด รูปภาพ และโค้ดฉบับเต็ม

จัดทำเพื่อประกอบโครงการ CUDA Hash Table
พฤศจิกายน 2026

บทคัดย่อ

บันทึกนี้สรุปสาระสำคัญของบทความใน NVIDIA Technical Blog ว่าด้วยการสร้างตารางแฮช (hash map) ที่ทำงานพร้อมกันได้จำนวนมากบน GPU โดยใช้ไลบรารี cuCollections เป็นกรณีศึกษา บทความชี้ว่าประสิทธิภาพของตารางแฮชถูกจำกัดโดยแบนด์วิดท์ของการเข้าถึงหน่วยความจำแบบสุ่ม (random memory access) เป็นหลัก และ GPU ในยุคปัจจุบันให้แบนด์วิดท์ลักษณะนี้สูงกว่า CPU ราวหนึ่งอันดับขนาด (order of magnitude) การออกแบบที่นำเสนอใช้การกำหนดที่อยู่แบบเปิด (open addressing) ร่วมกับการไล่ตรวจเชิงเส้น (linear probing) บนอาร์เรย์แบบแบน ใช้การดำเนินการอะตอมมิก (atomic operation) แทนการล็อก และใช้กลุ่มเธรดแบบร่วมมือ (cooperative groups) เพื่อให้หลายเธรดตรวจช่องข้างเคียงพร้อมกันด้วยการอ่านหน่วยความจำแบบรวม (coalesced load) ผลการทดลองบน H100 แสดงอัตราการแทรก 87.5 GB/s และการค้นหา 134.6 GB/s ซึ่งเร็วกว่าไลบรารีบน CPU มากกว่าสิบเท่า และเร็วกว่า Kokkos 3.8 และ 2.6 เท่าตามลำดับ ท้ายเอกสารได้รวบรวมข้อสังเกตเพื่อนำไปใช้ในโครงการพัฒนาตารางแฮชด้วย CUDA

สารบัญ

1 บทนำ **3**

2 พื้นฐานของตารางแฮช **3**

2.1 คอนเทนเนอร์เชิงสัมพันธ์ 3

2.2 การจัดเก็บ การค้นคืน และการชนกัน 3

2.3 การกำหนดที่อยู่แบบเปิดและการไล่ตรวจเชิงเส้น 3

3 การเข้าถึงหน่วยความจำแบบสุ่ม: ขีดจำกัดพื้นฐาน **4**

4 การออกแบบตารางแฮชบน GPU **4**

4.1 เป้าหมายและสมมติฐาน 4

4.2 การควบคุมการทำงานพร้อมกันโดยไม่ใช้ล็อก 5

5 กลุ่มเธรดแบบร่วมมือและการเข้าถึงหน่วยความจำ **6**

5.1 ปัญหาของการใช้หนึ่งเธรดต่อหนึ่งคีย์ 6

5.2 แนวทางแก้: ให้กลุ่มเธรดดูแลคีย์ร่วมกัน 6

5.3 ผลของการใช้กลุ่มเธรด 7

6 รูปแบบ API: การประมวลผลแบบกลุ่มและการเรียกจากฝั่งอุปกรณ์ **7**

7 ผลการวัดสมรรถนะเทียบกับไลบรารีอื่น **8**

7.1 วิธีการทดลอง 8

7.2 ผลลัพธ์ 8

7.3 กรณีข้อมูลอยู่ในหน่วยความจำของ CPU 8

8 กรณีศึกษา: การเชื่อมตารางหลายคอลัมน์ใน cuDF 8

9 ข้อแนะนำจากบทความ 9

10 ข้อสังเกตสำหรับการนำไปใช้ 9

11 บทสรุป 10

1 บทนำ

ตารางแฮชเป็นโครงสร้างข้อมูลพื้นฐานที่ให้การแทรกและการค้นหาในเวลาคงที่โดยเฉลี่ย (amortized $O(1)$) แม้จะใช้กันแพร่หลายบน CPU แต่การอภิปรายเรื่องตารางแฮชในบริบทของการประมวลผลบน GPU ยังมีไม่มากนัก บทความต้นฉบับจึงตั้งข้อเสนอหลักว่า GPU ซึ่งมีแบนด์วิดท์หน่วยความจำสูงและมีเธรดจำนวนมหาศาล เหมาะอย่างยิ่งกับงานที่ต้องแทรกหรือค้นหาข้อมูลจำนวนมากพร้อมกัน

ผู้เขียนแนะนำ **cuCollections** (`cuco`) ซึ่งเป็นไลบรารีโอเพนซอร์สภาษา CUDA C++ รวบรวมโครงสร้างข้อมูลแบบทำงานพร้อมกัน (concurrent data structures) รวมถึงตารางแฮชหลายรูปแบบ และยกตัวอย่างการใช้งานจริงใน RAPIDS cuDF เช่น การเชื่อมตาราง (join) และการคำนวณ TF-IDF นอกจากนี้ยังระบุงานประยุกต์อื่น ได้แก่ ระบบแนะนำ (recommender systems), การบีบอัดสตรีม (stream compaction), อัลกอริทึมกราฟ, จีโนมิกส์ และพีชคณิตเชิงเส้นแบบเบาบาง (sparse linear algebra)

บันทึกนี้จัดเรียงเนื้อหาใหม่เป็นลำดับ ได้แก่ พื้นฐานของตารางแฮช (หัวข้อ 2) ข้อจำกัดด้านหน่วยความจำ (หัวข้อ 3) การออกแบบบน GPU (หัวข้อ 4) กลุ่มเธรดแบบร่วมมือ (หัวข้อ 5) รูปแบบของ API (หัวข้อ 6) ผลการวัดสมรรถนะ (หัวข้อ 7) กรณีศึกษาการ join (หัวข้อ 8) ข้อแนะนำของผู้เขียน (หัวข้อ 9) และข้อสังเกตสำหรับการนำไปใช้ (หัวข้อ 10)

2 พื้นฐานของตารางแฮช

2.1 คอนเทนเนอร์เชิงสัมพันธ์

ตารางแฮชจัดอยู่ในกลุ่มคอนเทนเนอร์เชิงสัมพันธ์ (associative container) ซึ่งเก็บคู่ *คีย์–ค่า* (key–value pair) บทความใช้สมุดโทรศัพท์เป็นอุปมา คือชื่อบุคคลเป็นคีย์และหมายเลขโทรศัพท์เป็นค่า ผู้เขียนตั้งข้อสังเกตว่าใน C++ นั้น `std::map` มักสร้างจากต้นไม้ค้นหาแบบทวิภาค ส่วนที่ใกล้เคียงตารางแฮชจริงคือ `std::unordered_map` และในบทความใช้คำว่า hash table กับ hash map แทนกันได้

ตารางแฮชแบ่งได้เป็นสองแบบ คือ

- **แบบค่าเดียว** (single-value map) คีย์แต่ละตัวต้องไม่ซ้ำกัน
- **แบบหลายค่า** (multimap) อนุญาตให้คีย์ซ้ำได้ เช่น บุคคลหนึ่งมีหลายหมายเลข

2.2 การจัดเก็บ การค้นคืน และการชนกัน

ฟังก์ชันแฮช (hash function) แปลงคีย์เป็นดัชนีของช่องเก็บ (bucket) ในอาร์เรย์ การแทรกคือการวางคู่คีย์–ค่าลงในช่องนั้น ส่วนการค้นหาคือการคำนวณแฮชซ้ำแล้วไปอ่านช่องเดิม ในทางปฏิบัติไม่สามารถจัดให้คีย์หนึ่งตัวตรงกับช่องหนึ่งช่องได้เสมอ เพราะปริภูมิคีย์มักใหญ่เกินหรือไม่ทราบล่วงหน้า จึงเกิด**การชนกัน** (collision) เมื่อคีย์ต่างกันได้ดัชนีเดียวกัน ฟังก์ชันแฮชที่ดีช่วยลดแต่ไม่อาจขจัดการชนกันได้

2.3 การกำหนดที่อยู่แบบเปิดและการไล่ตรวจเชิงเส้น

บทความเลือกแก้ปัญหาการชนกันด้วย**การกำหนดที่อยู่แบบเปิด** (open addressing) ซึ่งเก็บทุกรายการไว้ในอาร์เรย์เดียว (ไม่ใช้ลิงก์ลิสต์ต่อท้ายแบบ separate chaining) เมื่อช่องเริ่มต้นถูกใช้แล้วจะเลื่อนไปยังช่องถัดไปตาม**ลำดับการไล่ตรวจ** (probing sequence) ซึ่งในบทความคือ**การไล่ตรวจเชิงเส้น** (linear probing) คือขยับ

ทีละหนึ่งช่องและวนกลับต้นอาร์เรย์เมื่อถึงปลาย สำหรับคีย์ k บนตารางขนาด C ช่อง ลำดับการไล่ตรวจเขียนได้เป็น

$$p_i(k) = (h(k) + i) \bmod C, \qquad i = 0, 1, 2, \ldots \tag{1}$$

ลำดับนี้กำหนดแน่นอนจากคีย์ การค้นหาจึงเดินตามลำดับเดียวกับการแทรก และหยุดได้ทันทีเมื่อพบช่องว่าง เพราะหมายความว่าคีย์นั้นไม่อยู่ในตาราง

ข้อดีของการไล่ตรวจเชิงเส้นคือเข้าถึงหน่วยความจำที่อยู่ติดกัน จึงเป็นมิตรกับแคช แต่ข้อเสียคือเมื่อ**อัตราส่วนการบรรจุ** (load factor) สูงขึ้น ลำดับการไล่ตรวจยาวขึ้นและต้องอ่านหน่วยความจำมากขึ้น

$$\alpha = \frac{\text{จำนวนช่องที่ถูกใช้}}{\text{จำนวนช่องทั้งหมด}} \tag{2}$$

ดังนั้นการเลือก α จึงเป็นการแลกเปลี่ยนระหว่างการใช้หน่วยความจำกับความเร็ว (หมายเหตุ: บทความกล่าวถึงเฉพาะการไล่ตรวจเชิงเส้น ส่วนรูปแบบอื่นเช่น double hashing มีใน cuCollections แต่ไม่ได้อภิปรายในบทความ)

3 การเข้าถึงหน่วยความจำแบบสุ่ม: ขีดจำกัดพื้นฐาน

เนื่องจากฟังก์ชันแฮชที่ดีกระจายคีย์ไปทั่วอาร์เรย์ รูปแบบการเข้าถึงตารางแฮชจึงเป็นแบบสุ่มโดยธรรมชาติ ผู้เขียนจึงใช้แบนด์วิดท์ของการอ่านแบบสุ่มเป็นตัวชี้วัดเพดานสมรรถนะ โดยวัดด้วยเกณฑ์มาตรฐาน GUPS (Giga-Updates Per Second) ซึ่งอ่านค่าขนาด 64 บิตจากตำแหน่งสุ่ม ผลสรุปได้ดังตารางที่ 1

ตารางที่ 1: แบนด์วิดท์ทางทฤษฎีเทียบกับแบนด์วิดท์การอ่านแบบสุ่มที่วัดได้ (เรียบเรียงจากข้อมูลในบทความ)

หน่วยประมวลผล (หน่วยความจำ)	**สูงสุดทางทฤษฎี (GB/s)**	**อ่านแบบสุ่ม (GB/s)**
Intel Xeon Platinum 8360Y (DDR4-3200, 8 ช่อง)	204	15
NVIDIA A100-80GB-SXM (HBM2e)	2039	141
NVIDIA H100-80GB-SXM (HBM3)	3352	256

ข้อสังเกตสำคัญมีสองประการ ประการแรก การอ่านแบบสุ่มได้แบนด์วิดท์ต่ำกว่าค่าทางทฤษฎีราวสิบเท่าในทุกแพลตฟอร์ม เพราะระบบหน่วยความจำถูกออกแบบมาเพื่อการอ่านต่อเนื่อง ประการที่สอง GPU ให้แบนด์วิดท์แบบสุ่มสูงกว่า CPU ราวหนึ่งอันดับขนาด ผู้เขียนจึงสรุปว่าตารางแฮชที่ดีที่สุดบน CPU น่าจะช้ากว่าตารางแฮชที่ดีที่สุดบน GPU ประมาณสิบเท่า

4 การออกแบบตารางแฮชบน GPU

4.1 เป้าหมายและสมมติฐาน

ตารางแฮชบน GPU ในบทความ*ไม่ได้*มุ่งเป็นตัวแทนของ `std::unordered_map` แต่มุ่งรองรับงานที่มีปริมาณงานสูงและขนานมาก ตัวอย่างที่ใช้อธิบายตั้งสมมติฐานเพื่อความเรียบง่ายดังนี้

1. ความจุคงที่ ไม่ขยายขนาดภายหลัง
2. สงวนค่าคีย์หนึ่งค่าไว้เป็น**ค่าเฝ้าระวังว่าง** (empty sentinel) เพื่อระบุช่องที่ยังไม่ถูกใช้

3. ขนาดคีย์รวมกับขนาดค่าไม่เกิน 8 ไบต์ เพื่อให้ทั้งคู่ถูกเขียนได้ด้วยอะตอมมิกเพียงครั้งเดียว
4. ไม่รองรับการลบ

ผู้เขียนระบุว่า cuCollections มีโครงสร้างที่ซับซ้อนกว่าซึ่งผ่อนคลายข้อจำกัดเหล่านี้

4.2 การควบคุมการทำงานพร้อมกันโดยไม่ใช้ล็อก

แต่ละช่องในตารางเป็นตัวแปรอะตอมมิกของคู่คีย์–ค่า (จาก libcu++ คือ `cuda::std::atomic`) การแทรกจึงทำได้โดยไม่ต้องล็อก (lock-free) ผ่านการเปรียบเทียบแล้วสลับ (compare-and-swap, CAS) โดยคาดหวังว่าช่องนั้นยังเท่ากับค่าเฝ้าระวังว่าง ขั้นตอนเชิงตรรกะสรุปได้ดังนี้

1. คำนวณช่องเริ่มต้นจากแฮชของคีย์
2. อ่านช่อง หากว่างให้พยายาม CAS เพื่อเขียนคู่คีย์–ค่า หากสำเร็จถือว่าแทรกเสร็จ
3. หากช่องมีคีย์เดียวกันอยู่แล้ว รายงานว่าซ้ำ (สำหรับตารางแบบค่าเดียว)
4. มิฉะนั้น (รวมถึงกรณี CAS แพ้เธรดอื่น) เลื่อนไปช่องถัดไปแบบวนรอบ

รายการโค้ดที่ 1 เป็นตัวอย่างที่ผู้จัดทำเขียนขึ้นเองเพื่อแสดงหลักการ (ไม่ได้คัดลอกจากบทความ) โดยบีบคีย์และค่าขนาด 32 บิตรวมเป็นคำขนาด 64 บิตหนึ่งคำ

รายการโค้ดที่ 1: การแทรกแบบหนึ่งเธรดต่อหนึ่งคีย์ด้วย CAS (ตัวอย่างเพื่อการศึกษา)

```
constexpr uint32_t EMPTY_KEY = 0xFFFFFFFFu;
constexpr uint64_t EMPTY_SLOT = 0xFFFFFFFFFFFFFFFFull;

__device__ inline uint64_t pack(uint32_t k, uint32_t v) {
  return (uint64_t(k) << 32) | v;
}

// returns true if inserted, false if key already present
__device__ bool insert(unsigned long long* slots, size_t capacity,
                       uint32_t key, uint32_t value) {
  size_t idx = murmur3_32(key) % capacity;      // any good hash
  for (size_t step = 0; step < capacity; ++step) {
    unsigned long long cur = slots[idx];
    if (cur == EMPTY_SLOT) {
      unsigned long long old =
          atomicCAS(&slots[idx], EMPTY_SLOT, pack(key, value));
      if (old == EMPTY_SLOT) return true;        // won the slot
      cur = old;                                 // lost the race
    }
    if (uint32_t(cur >> 32) == key) return false; // duplicate
    idx = (idx + 1) % capacity;                  // linear probing
  }
  return false;                                  // table full
}
```

5 กลุ่มเธรดแบบร่วมมือและการเข้าถึงหน่วยความจำ

5.1 ปัญหาของการใช้หนึ่งเธรดต่อหนึ่งคีย์

ในรูปแบบตามรายการโค้ดที่ 1 เธรด 32 ตัวใน warp เดียวกันจะประมวลผลคีย์ที่แฮชไปคนละตำแหน่ง ในแต่ละรอบของการไล่ตรวจ warp จึงอาจแตะหน่วยความจำมากถึง 32 บริเวณที่ไม่เกี่ยวข้องกัน อีกทั้งการไล่ตรวจเชิงเส้นของเธรดเดียวจะอ่านช่องติดกันทีละช่อง ทำให้ไม่สามารถดึงช่องข้างเคียงล่วงหน้า ด้วยการอ่านแบบรวมครั้งเดียวได้ ปัญหานี้รุนแรงขึ้นเมื่ออัตราส่วนการบรรจุสูงและลำดับการไล่ตรวจยาว

5.2 แนวทางแก้: ให้กลุ่มเธรดดูแลคีย์ร่วมกัน

บทความเสนอให้ใช้ CUDA cooperative groups แบ่ง warp ออกเป็นกลุ่มย่อยขนาด g เธรดที่อยู่ติดกัน (ตัวอย่างในบทความใช้ $g = 4$) โดยแต่ละกลุ่มรับผิดชอบคีย์เพียงตัวเดียว กลไกเป็นดังนี้

- กลุ่มอ่าน**หน้าต่างการไล่ตรวจ** (probing window) ขนาด g ช่องติดกันด้วยการอ่านแบบรวมครั้งเดียว โดยแต่ละเธรดรับผิดชอบหนึ่งช่อง
- ใช้ `ballot` ตรวจว่ามีเธรดใดพบคีย์เดียวกันแล้วหรือไม่ และมีช่องว่างที่ใดบ้าง
- เลือกเธรดอันดับต่ำสุดที่พบช่องว่างเป็นผู้แทน (leader) ให้ทำ CAS
- กระจายผลลัพธ์ (สำเร็จ ซ้ำ หรือต้องไปต่อ) กลับไปทั้งกลุ่มด้วย `shuffle`
- หากหน้าต่างไม่มีช่องว่าง กลุ่มเลื่อนไปยังหน้าต่างถัดไป

ผลคือจำนวนรอบการไล่ตรวจลดลงราว g เท่า และการอ่านแต่ละครั้งเป็นแบบรวม แลกกับการที่จำนวนคีย์ที่ประมวลผลพร้อมกันต่อ warp ลดลง ผู้เขียนระบุว่าโค้ดตัวอย่างในบทความเป็นฉบับย่อของ การทำงานจริงใน `cuco::static_map`

รายการโค้ดที่ 2: โครงร่างการแทรกแบบกลุ่มเธรด (ตัวอย่างเพื่อการศึกษา; ละรายละเอียดบางส่วน)

```
namespace cg = cooperative_groups;

template <int G>
__device__ bool cg_insert(cg::thread_block_tile<G> tile,
                          unsigned long long* slots, size_t capacity,
                          uint32_t key, uint32_t value) {
  size_t base = (murmur3_32(key) % capacity) & ~size_t(G - 1);
  while (true) {
    size_t idx = (base + tile.thread_rank()) % capacity;
    unsigned long long cur = slots[idx];           // coalesced window load
    if (tile.any(uint32_t(cur >> 32) == key)) return false;
    unsigned empty = tile.ballot(cur == EMPTY_SLOT);
    if (empty) {
      int leader = __ffs(empty) - 1;               // lowest empty lane
      bool ok = false;
      if (tile.thread_rank() == leader)
        ok = atomicCAS(&slots[idx], EMPTY_SLOT, pack(key, value))
             == EMPTY_SLOT;
      if (tile.shfl(ok, leader)) return true;
      continue;                                    // re-read same window
```

```
    }
    base = (base + G) % capacity;                    // next window
  }
}
```

5.3 ผลของการใช้กลุ่มเธรด

ผลการทดลองบน A100 80GB ในบทความแสดงว่า

- ที่อัตราส่วนการบรรจุต่ำ แบบไม่ร่วมมือ (หนึ่งเธรดต่อคีย์) ทำได้ใกล้ค่าเหมาะสมที่สุดอยู่แล้ว
- ที่อัตราส่วนการบรรจุสูง แบบไม่ร่วมมือมีปริมาณงานลดลงมากเพราะการชนกันและลำดับการไล่ตรวจที่ยาว
- เมื่อใช้กลุ่มขนาด 4 การแทรกเร็วขึ้นได้ถึงราว 13% และการค้นหาเร็วขึ้นได้ถึงราว 40% ในช่วงอัตราส่วนการบรรจุสูง
- ประโยชน์ยังขยายไปถึงตารางแบบหลายค่าที่คีย์ซ้ำกันมาก เพราะคีย์ซ้ำต้องเดินลำดับช่องเดียวกัน

ค่า GUPS ในหัวข้อ 3 ถูกใช้เป็นเส้นขอบบนของกราฟผลลัพธ์ เพื่อแสดงว่าการทำงานเข้าใกล้ขีดจำกัดฮาร์ดแวร์เพียงใด

6 รูปแบบ API: การประมวลผลแบบกลุ่มและการเรียกจากฝั่งอุปกรณ์

บทความวัดสมรรถนะด้วยการดำเนินการแบบกลุ่ม (bulk operation) คือแทรกหรือค้นหาคีย์จำนวนมากในการเรียกครั้งเดียว แต่ไม่ได้อภิปรายการออกแบบ API โดยละเอียด เพื่อความครบถ้วน ผู้จัดทำขอสรุปจากเอกสาร cuCollections ว่ามีการใช้งานสองระดับ

- **API แบบกลุ่มจากฝั่งโฮสต์** (bulk host API) เช่น `insert` และ `find` ที่รับช่วงข้อมูล (iterator range) บนหน่วยความจำ GPU แล้วเรียกเคอร์เนลภายในให้ เหมาะกับงานแบบ “สร้างทั้งตารางแล้วค้นทั้งชุด” และได้ประโยชน์จากการปรับแต่งภายในไลบรารีเต็มที่
- **API ฝั่งอุปกรณ์** (device-side API) ซึ่งส่งวัตถุอ้างอิง (reference/view) ของตารางเข้าไปในเคอร์เนลของผู้ใช้ ให้เรียกแทรกหรือค้นหาได้ภายในเคอร์เนลเอง เหมาะเมื่อการเข้าถึงตารางผสานอยู่กับตรรกะอื่น เช่นในขั้น probe ของการ join

รายการโค้ดที่ 3: ภาพรวมการใช้ API แบบกลุ่ม (สไตล์ cuCollections; ชื่อพารามิเตอร์อาจต่างตามรุ่น)

```
// 2^27 pairs at target load factor 0.5 -> capacity about 2^28
auto map = cuco::static_map{capacity,
                            cuco::empty_key{-1}, cuco::empty_value{-1}};
map.insert(pairs.begin(), pairs.end());              // bulk insert
map.find(keys.begin(), keys.end(), results.begin());  // bulk find
```

7 ผลการวัดสมรรถนะเทียบกับไลบรารีอื่น

7.1 วิธีการทดลอง

ผู้เขียนเทียบ `cuco::static_map` กับไลบรารีสามกลุ่ม ได้แก่

1. แบบลำดับบน CPU: `std::unordered_map` และ `absl::flat_hash_map` (ต้องซิงโครไนซ์เองหากใช้หลายเธรด)
2. แบบทำงานพร้อมกันบน CPU: `tbb::concurrent_hash_map` และ `folly::AtomicHashMap`
3. บน GPU: `Kokkos::UnorderedMap`

งานทดสอบคือแทรกคู่คีย์–ค่าที่ไม่ซ้ำจำนวน 2^{27} คู่ (คีย์ 4 ไบต์ ค่า 4 ไบต์ รวมประมาณ 1 GB) ที่อัตราส่วนการบรรจุเป้าหมาย 50% แล้วค้นหาคีย์ชุดเดิม โดยรายงานผลเป็นปริมาณงานด้านหน่วยความจำ (GB/s)

7.2 ผลลัพธ์

ตารางที่ 2: สรุปตัวเลขสำคัญจากการทดลองบน NVIDIA H100-80GB-SXM หนึ่งตัว

รายการ	แทรก (insert)	ค้นหา (find)
`cuco::static_map` (GB/s)	87.5	134.6
อัตราเร่งเทียบ `Kokkos::UnorderedMap`	3.8 เท่า	2.6 เท่า
เทียบไลบรารี CPU ที่เร็วที่สุด	เร็วกว่ามากกว่าหนึ่งอันดับขนาด	

ตัวเลขของไลบรารีบน CPU แต่ละตัวแสดงไว้ในรูปกราฟของบทความเท่านั้น จึงไม่ได้ระบุค่าละเอียดในที่นี้ ข้อสังเกตคือการค้นหาเร็วกว่าการแทรกเพราะไม่ต้องใช้ CAS และการเปรียบเทียบนี้สอดคล้องกับผลในตารางที่ 1 ที่ทำนายช่องว่างราวสิบเท่าระหว่าง CPU กับ GPU

7.3 กรณีข้อมูลอยู่ในหน่วยความจำของ CPU

การเปรียบเทียบข้างต้นยุติธรรมเฉพาะเมื่อข้อมูลอยู่บน GPU แล้ว หากข้อมูลเริ่มต้นอยู่ฝั่ง CPU ต้องส่งผ่าน PCIe ไม่ว่าจะด้วยการคัดลอกแบบอะซิงโครนัสหรือการย้ายหน้าของหน่วยความจำรวม (unified memory) ผู้เขียนให้เหตุผลว่าปริมาณงานของ cuCollections สูงกว่าแบนด์วิดท์ที่ใช้ได้จริงของ PCIe Gen4 (และ Gen5 สำหรับ H100) ดังนั้นตารางแฮชบน GPU ยังคงทำงานได้เต็มความเร็วของลิงก์ระหว่าง CPU กับ GPU และระบบ Grace Hopper ที่ใช้ NVLink-C2C จะยิ่งเพิ่มอัตราเร่งได้อีก

8 กรณีศึกษา: การเชื่อมตารางหลายคอลัมน์ใน cuDF

RAPIDS cuDF ใช้ตารางแฮชของ cuCollections ในการทำ hash join แบบหลายคอลัมน์ ซึ่งแบ่งเป็นสองขั้น

1. **ขั้นสร้าง** (build) คำนวณแฮชของทั้งแถวในตารางหนึ่ง แล้วแทรกลงใน `cuco::static_multimap` โดยใช้ค่าแฮชของแถวเป็นคีย์และดัชนีแถวเป็นค่า (ต้องเป็น multimap เพราะหลายแถวอาจมีค่าแฮชเท่ากัน)

2. **ขั้นค้น** (probe) คำนวณแฮชของแต่ละแถวในอีกตาราง แล้วดึงดัชนีแถวทั้งหมดที่มีแฮชตรงกัน

เนื่องจากค่าแฮชที่เท่ากันไม่ได้รับประกันว่าแถวเท่ากัน ทุกคู่ผู้สมัครจึงต้องผ่าน**ฟังก์ชันตรวจความเท่ากันของแถว** (row equality) อีกชั้น ตัวอย่างในบทความแสดงกรณีที่สองแถวฝั่งซ้ายมีแฮชตรงกับแถวฝั่งขวา แต่มีเพียงแถวเดียวที่ผ่านการตรวจความเท่ากันและนับเป็นคู่ที่ join ได้จริง บทความไม่ได้รายงานตัวเลขสมรรถนะของ join แต่อ้างถึงงานนำเสนอเรื่อง operator ฐานข้อมูลบน GPU และ join แบบหลาย GPU

9 ข้อแนะนำจากบทความ

ผู้เขียนสรุปหลักการออกแบบตารางแฮชบน GPU ที่มีประสิทธิภาพไว้สองข้อ

1. ใช้**โครงสร้างหน่วยความจำแบบแบน** (flat memory layout) ของช่องเก็บ ร่วมกับการกำหนดที่อยู่แบบเปิด
2. ให้**เธรดหลายตัวร่วมมือกัน**ตรวจช่องข้างเคียงทั้งในการแทรกและการค้นหา ซึ่งให้ผลชัดเจนเมื่ออัตราส่วนการบรรจุสูง

และให้ข้อสรุปว่า GPU เหมาะกับโครงสร้างข้อมูลแบบทำงานพร้อมกันเพราะมีหน่วยความจำแบนด์วิดท์สูง รองรับการอ่านขนาดเล็กแบบสุ่มและการดำเนินการอะตอมมิกได้รวดเร็ว พร้อมแนะนำให้ใช้ cuCollections ในงานจริงแทนการเขียนเอง

10 ข้อสังเกตสำหรับการนำไปใช้

หัวข้อนี้เป็นความเห็นของผู้จัดทำสำหรับโครงการพัฒนาตารางแฮชด้วย CUDA โดยต่อยอดจากเนื้อหาของบทความ

1. **ตั้งเส้นฐานด้วย GUPS ก่อน** ควรวัดแบนด์วิดท์การอ่านแบบสุ่มบน GPU ที่ใช้จริง แล้วรายงานผลตารางแฮชเป็นร้อยละของค่านี้ จะประเมินได้ว่ายังเหลือช่องว่างให้ปรับปรุงเท่าใด
2. **ออกแบบช่องให้เขียนด้วยอะตอมมิกครั้งเดียว** หากคีย์+ค่าเกิน 8 ไบต์ ต้องพิจารณา CAS แบบ 128 บิต (บนสถาปัตยกรรมที่รองรับ) หรือเขียนคีย์ก่อนแล้วจึงเขียนค่า ซึ่งต้องจัดการลำดับการมองเห็น (memory ordering) อย่างระมัดระวัง
3. **เลือกค่าเฝ้าระวังให้ไม่ชนกับข้อมูลจริง** และตรวจสอบอินพุตว่าไม่มีคีย์เท่ากับค่าเฝ้าระวัง
4. **กำหนดอัตราส่วนการบรรจุอย่างมีเหตุผล** ช่วงประมาณ 0.5 เป็นจุดตั้งต้นที่ดีสำหรับความเร็ว หากต้องประหยัดหน่วยความจำและใช้ α สูง ควรใช้กลุ่มเธรดแบบร่วมมือ
5. **ทดลองขนาดกลุ่ม g หลายค่า** (1, 2, 4, 8) เพราะค่าที่ดีที่สุดขึ้นกับขนาดช่อง อัตราส่วนการบรรจุ และสัดส่วนระหว่างการแทรกกับการค้นหา
6. **ใช้ฟังก์ชันแฮชคุณภาพดี** เช่น MurmurHash3 หรือ xxHash เพราะการไล่ตรวจเชิงเส้นไวต่อการเกาะกลุ่ม (clustering) ของคีย์ และควรเลือกความจุเป็นกำลังสองเพื่อแทนการหารเอาเศษด้วยการ AND บิต
7. **การลบ** หากโครงการต้องรองรับการลบ ต้องเพิ่มค่าเฝ้าระวังสำหรับช่องที่ถูกลบ (tombstone) เพื่อไม่ให้การค้นหาหยุดก่อนเวลาอันควร ซึ่งบทความไม่ได้ครอบคลุม
8. **เทียบกับ cuCollections เสมอ** ใช้ `cuco::static_map` เป็นเส้นฐานเชิงปฏิบัติ และวัดเวลาแยกส่วนระหว่างการถ่ายโอนผ่าน PCIe กับการคำนวณบน GPU

11 บทสรุป

บทความแสดงให้เห็นว่าสมรรถนะของตารางแฮชถูกกำหนดโดยแบนด์วิดท์การเข้าถึงหน่วยความจำแบบสุ่ม ซึ่ง GPU เหนือกว่า CPU ราวสิบเท่า การออกแบบที่ใช้อาร์เรย์แบน การกำหนดที่อยู่แบบเปิดด้วยการไล่ตรวจเชิงเส้น การเขียนแบบอะตอมมิกโดยไม่ใช้ล็อก และกลุ่มเธรดแบบร่วมมือ ช่วยให้ cuCollections เข้าใกล้ขีดจำกัดของฮาร์ดแวร์ ได้ผลลัพธ์ 87.5 และ 134.6 GB/s สำหรับการแทรกและการค้นหาบน H100 และนำไปใช้จริงแล้วในงานอย่าง hash join ของ cuDF สำหรับโครงการพัฒนาตารางแฮชเอง บทเรียนสำคัญคือให้วัดเทียบกับเพดาน GUPS ออกแบบช่องให้เหมาะกับอะตอมมิก และใช้การร่วมมือของเธรดเมื่อทำงานที่อัตราส่วนการบรรจุสูง

เอกสารอ้างอิง

[1] D. Juenger, N. Iskos, Y. Wang, J. Hemstad, C. Hundt, and N. Sakharnykh, “Maximizing Performance with Massively Parallel Hash Maps on GPUs,” *NVIDIA Technical Blog*, 6 Mar. 2023. [Online]. Available: `https://developer.nvidia.com/blog/maximizing-performance-with-massively-parallel-hash-maps-on-gpus/` (สืบค้นเมื่อ 3 พฤศจิกายน 2026)

[2] NVIDIA, “cuCollections: Concurrent data structures for CUDA,” GitHub repository. [Online]. Available: `https://github.com/NVIDIA/cuCollections`

[3] RAPIDS, “cuDF: GPU DataFrame Library,” GitHub repository. [Online]. Available: `https://github.com/rapidsai/cudf`

[4] NVIDIA, “CUDA C++ Programming Guide: Cooperative Groups.” [Online]. Available: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`

[5] NVIDIA, “libcu++: `cuda::std::atomic`.” [Online]. Available: `https://nvidia.github.io/libcudacxx/extended_api/synchronization_primitives/atomic.html`

[6] Innovative Computing Laboratory, “HPC Challenge: RandomAccess (GUPS).” [Online]. Available: `https://icl.utk.edu/projectsfiles/hpcc/RandomAccess/`